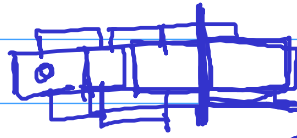
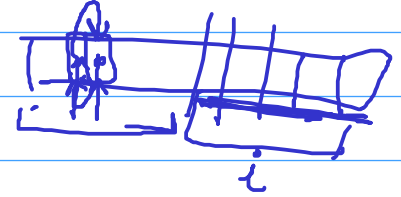


① Bubble Sort

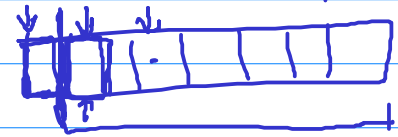


Best
 $O(n)$

```
for(i=0; i<n-1; i++) {
    int swap_cnt=0;
    for(j=0; j<n-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
            swap_cnt++;
        }
        if(swap_cnt == 0) break;
    }
}
```



2 - 0 - 1
①

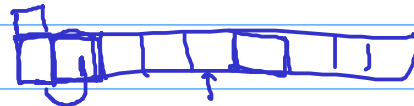


② selection

```
for(i=0; i<n-1; i++) {
    min_index = i;
    for(j=i+1; j<n; j++) {
        if(arr[j] < arr[min_index])
            min_index = j;
    }
    swap(arr[i], arr[min_index]);
}
```

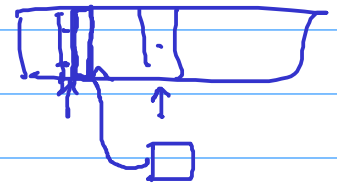
3

③ Insertion sort



$O(n^2)$

```
for(i=1; i<n; i++) {
    int element_to_insert = arr[i];
    for(j=i-1; j>=0; j--) {
        if(arr[j] > element_to_insert)
            arr[j+1] = arr[j];
        else {
            arr[j+1] = element_to_insert;
            break;
        }
    }
}
```



$O(n)$

$O(n)$ - Best case

3

①

	Time Complexity			Space complexity (Excluding array)
	Best	avg	worst	
Bubble	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
→ Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	worst case : $O(n)$ ✓ best/avg case : $O(\log n)$

↑ (proof)